

Low-Code, High Impact

Weaponizing n8n for Automated Offensive Security

Veronika Stoyanova
Cyber Security Automation



One self-hosted n8n workflow that turns **11 open-source tools + AI** into a single pipeline - from a webhook to a finished vulnerability report.

[Live Demo](#)

[AI Security Analyst](#)

[11 Security tools](#)

Low-code is a legitimate offensive-tooling layer

\$_WHOAMI · THE THESIS

Who's talking

1

Cyber Security Automation | Penetration Tester

offensive security by trade — MSc Cyber Security & Digital Forensics

2

Co-founder, OpsIntelGuard

we build security automations for SMBs

3

Why this talk

the same orchestration that runs a sales funnel can run a kill-chain

This talk

IS

a real architecture: logic, nodes, code, execution

IS

honest — including the bugs and the hacks

IS NOT

a vendor pitch or a slideware demo

IS NOT

a how-to for attacking systems you don't own

Scope & ethics first - this is a lab

RULES OF ENGAGEMENT

Authorized target only

	Target	DVWA - Damn Vulnerable Web Application, on purpose
	Location	isolated Kali VM at 192.168.68.130 on a local network
	Authorization	my own lab, my own hardware - no third-party systems
	Production caveat	the real opsintelguard.com is only recon-scanned, ahead of time

Why a vulnerable target

- ▶ Every tool actually fires and finds real issues
- ▶ Findings are reproducible for a live audience
- ▶ SQLi / headers / TLS / CVEs all present by design
- ▶ Aggressive tools (Gobuster, SQLmap) can't hurt anyone

n8n as an orchestration layer

THE IDEA

n8n = self-hosted, node-based workflow automation. **Each tool becomes a node; the glue is JavaScript.**

Nodes, not scripts

Every tool = a node. The DAG is the program — visible, reorderable, debuggable step-by-step.

Branching & retries free

IF nodes, parallel fan-out, Retry-On-Fail, On-Error-Continue — no boilerplate.

SSH does the real work

n8n runs on a server; each tool node SSHes into Kali and runs the actual CLI.

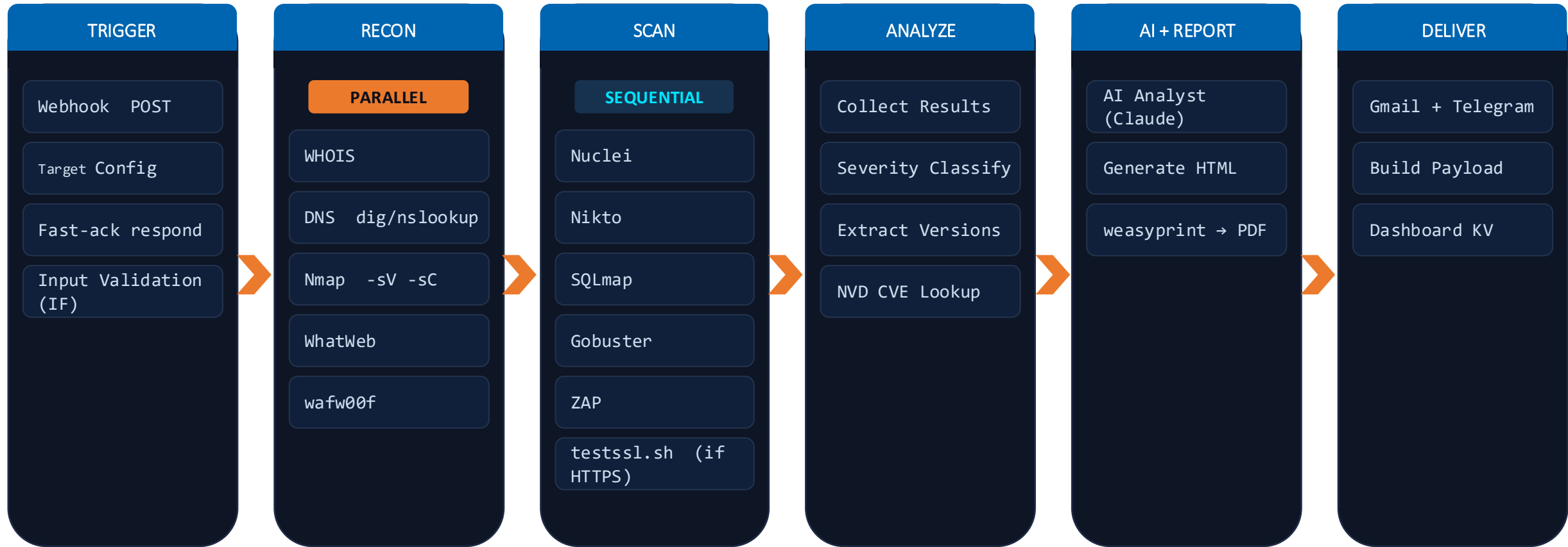
AI is a first-class node

A LangChain Agent node drops Claude into the pipeline like any other step.

n8n orchestrates › Kali executes › Claude explains

One webhook in, a report + live dashboard out

ARCHITECTURE · THE WHOLE PIPELINE



Six decisions that shaped the build

DESIGN PHILOSOPHY

1

Parallel recon, sequential scan

Recon is independent & cheap → fan out.
Scanning is heavy & order-sensitive → chain it.

2

Fail-open, never fail-hard

safeGet + On-Error-Continue: one dead tool degrades a report, it doesn't kill the run.

3

Fast-ack the caller

Respond to the webhook in <1s, then keep scanning - or the edge kills the connection.

4

Telemetry must not block

Dashboard reporting nodes retry & continue-on-error; the scan never waits on them.

5

AI analyses, tools detect

Claude is a reasoning layer on raw output - it never replaces a scanner.

6

One JSON is the contract

Every stage reads/writes one structured payload; the dashboard & report consume it.

Entry point - Webhook → Target Configuration

STAGE 1 • TRIGGER

A POST to [/webhook/owasp-vuln-scan](#) starts everything. A Set node normalises the request into the payload every later node reads.

```
Target Configuration – Set node

scan_id    = {{ $now.format('yyyyMMdd_HH:mm:ss') }}
target_ip  = {{ $json.body.target_ip }}
target_url = {{ $json.body.target_url }}
scan_type  = {{ $json.body.scan_type }}
ports      = {{ $json.body.ports }}
output_dir = /tmp/owasp_scans
started_at = {{ $now.toISO() }}
```

Why a Set node here

scan_id

one timestamp keys the output dir, the report file and the dashboard record

started_at

captured now so the final node can compute real scan duration

one source of truth

every downstream node reads \$('Target Configuration'), never the raw body

Surviving Cloudflare Tunnel: the fast-ack pattern

STAGE 1 · WAR STORY

```
Respond to Webhook Connection

// Respond to Webhook Connection – fires right after Target Config
// Respond With must be TEXT, not JSON (see below)
responseBody =
  {{ JSON.stringify({
    status: "accepted",
    scan_id: $('Target Configuration').first().json.scan_id,
    target: $('Target Configuration').first().json.target_ip,
    message: "Scan initiated"
  }) }}
```

⚠ two traps

~100s idle timeout - a full scan takes 17–20 min; the tunnel cuts an open connection long before it finishes. So ack in <1s, then keep working.

JSON mode rejects it - n8n's JSON responder re-validates {{ {...} }} and errors on valid object expressions. Use Text + JSON.stringify.

The pattern

Webhook → Target Config → **Respond (ack)** → Input Validation → ... 20-minute pipeline continues in the background

Validate before you spend 20 minutes

STAGE 1 · GATING

```
Input Validation - SSH
echo "=== TARGET VALIDATION ===" && \
ping -c 2 -W 3 {{ $json.target_ip }} 2>&1 && \
curl -s -o /dev/null -w "HTTP_STATUS:%{http_code}" \
--max-time 5 {{ $json.target_url }} 2>&1 && \
echo "VALIDATION:PASSED" || echo "VALIDATION:FAILED"
```

Validation Check (IF)

PASSED → scan pipeline

FAILED → Telegram alert

...then respond to the caller and stop.

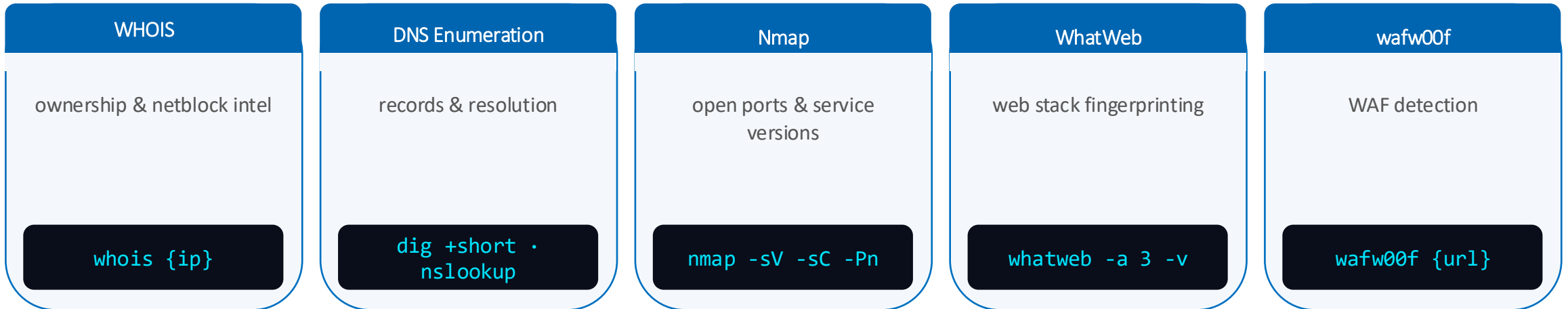
Fail-fast is a design principle, not politeness

A dead host or wrong URL should cost a few seconds, not a 20-minute scan against nothing. Two cheap checks — ICMP reachable and HTTP responding — gate the entire expensive pipeline behind one IF node.

Parallel fan-out - five tools at once

STAGE 2 · RECONNAISSANCE

Create **Output Directory** fans out to five independent SSH nodes that all run at the same time, then **Wait-for-All** (Merge) re-joins them.



Why parallel here?

These tools don't depend on each other and each is light. Running them concurrently turns five sequential waits into one - wall-clock time drops to the slowest single tool instead of their sum.

Anatomy of a tool node - Nmap over SSH

STAGE 2 · NODE ANATOMY

```
Nmap - SSH node

nmap -sV -sC -Pn \
  -p {{ $('Target Configuration').item.json.ports }} \
    {{ $('Target Configuration').item.json.target_ip }} \
  -oN {{ $out }}/{{ $scan_id }}/nmap.txt 2>&1
```

The pattern, x11

All eleven tool nodes are this same shape: an SSH node that interpolates the shared config with {{ }} expressions, runs the real CLI on Kali, writes a file, and returns stdout. Learn one, you've read them all.

Reading the flags

-sV	service / version detection	-sC	default NSE scripts
-Pn	skip host discovery (we know it's up)	-oN	normal output to a file we'll read back

Sequential chain + a conditional branch

STAGE 3 · SCANNING

Nuclei

Nikto

SQLmap

Gobuster

ZAP

IF HTTPS?

true → testssl.sh

false → skip

```
SQLmap - SSH node

sqlmap -u "{{ $url }}/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=test; security=low" \
--batch --risk=2 --level=3 \
--output-dir="{{ $out }}/{{ $scan_id }}/sqlmap 2>&1 | tail -50
```

Why sequential?

- ▶ Heavy, noisy tools — don't hammer the target in parallel
- ▶ Some steps build on earlier discovery
- ▶ --batch = non-interactive; answers every prompt
- ▶ tail -50 keeps only the useful verdict from a huge log

Collect Results - defensive aggregation

STAGE 4 • AGGREGATION

```
Collect Results - Code node

const safeGet = (fn) => {
  try { return fn() || 'No data'; }
  catch (e) { return 'Not executed'; } // a dead tool never kills the run
};

const nmap = safeGet(() => $('Nmap').first().json.stdout);
const nuclei = safeGet(() => $('Nuclei').first().json.stdout);
const sqlmap = safeGet(() => $('SQLmap').first().json.stdout);
// ...all 11 tools folded into ONE structured JSON payload
return [{ json: { nmap, nuclei, sqlmap, /* ... */ full_report } }];
```

Principle: fail-open

Any single tool can crash, time out, or return nothing. Wrapping every read in `safeGet` means a missing result becomes a label - 'Not executed' - instead of an exception that aborts the whole pipeline.

A partial report always beats a crashed run.

Severity Classification - the triage engine

STAGE 4 · TRIAGE

```
Severity Classification - Code node

const strip = s => s.replace(/\x1B\[([0-9;]*[a-zA-Z])/g, ''); // kill ANSI
const noise = [/^starting/i, /templates loaded/i, /Status: \d+/, ...];

for (const line of lines) {
  if (noise.some(p => p.test(line))) continue; // drop banners
  if (/rce|command injection|buffer overflow/.test(l)) critical.push(line);
  else if (/sql injection|xss|traversal|\[high\]/.test(l)) high.push(line);
  else if (/misconfig|header missing|weak cipher/.test(l)) medium.push(line);
  else if (/outdated|cve-|server banner/.test(l)) low.push(line);
}
```

Pipeline inside

1. Strip ANSI colour codes
2. Filter scanner banner noise
3. Tier by keyword → C/H/M/L
4. Deduplicate findings
5. Map each → OWASP Top 10

Honest limitation

This is keyword matching, not semantics — it over- and under-classifies. That's exactly why the next two stages exist: real CVE data from NVD, and an AI layer that reads context.

From version strings to real CVEs (NVD)

STAGE 4 • ENRICHMENT

```
NVD CVE Lookup — Code node

for (const sw of detected_software) {           // e.g. "Apache 2.4.49"
  const url = `.../cves/2.0?keywordSearch=${sw.query}&resultsPerPage=3`;
  const res = await this.helpers.httpRequest({ method: 'GET', url,
    headers: { 'User-Agent': 'OWASP-Scanner/1.0' }, timeout: 10000 });

  const score = cvssV31.baseScore || cvssV2.baseScore || 0; // v3.1 → v2 fallback
  cveResults.push({ software: sw.software, version: sw.version, cves });

  await new Promise(r => setTimeout(r, 2000)); // respect NVD rate limit
}
```

What it adds

Extract Versions regexes Apache / PHP / OpenSSL / jQuery... out of nmap + WhatWeb output.

Then each version is looked up against the live NVD database — turning "Apache 2.4.49" into concrete CVE IDs with CVSS scores.

Three robustness details: async httpRequest per item · CVSS v3.1→v2 fallback · 2s sleep so NVD doesn't rate-limit us

AI Security Analyst - prompt engineering as code

STAGE 5 • THE REASONING LAYER

AI Security Analyst – system message

You are an expert penetration tester and cybersecurity analyst.
Your report MUST include:

- ## Executive Risk Assessment
- ## Critical Recommendations (what · why · exact fix)
- ## Software Update Priorities (table + CVE references)
- ## Quick Wins (within 24 hours)
- ## OWASP Top 10 (2021) Mapping
- ## Compliance Notes (PCI-DSS · ISO 27001 · GDPR)

Rules: be specific · exact versions & commands · prioritize by real-world exploitability · flag public exploits · no fluff.

How it's wired

LangChain Agent node

Claude Sonnet (Anthropic)

The user prompt injects the severity summary, detected software, CVEs and the raw Critical/High/Medium findings. The system message forces a consistent, structured report.

The AI never scans. It reads structured tool output and turns findings into a prioritised, remediation-ready report.

Two OWASP mappings that disagree

STAGE 5 · A NUANCE WORTH SHOWING

Deterministic · mapToOwasp()

A pure keyword function in the Code node. Fast, repeatable, auditable — but literal.

```
permissions-policy missing → A05 Security Misconfiguration
```

Because 'header' → A05 by rule, regardless of the specific header's intent.

Contextual · the AI analyst

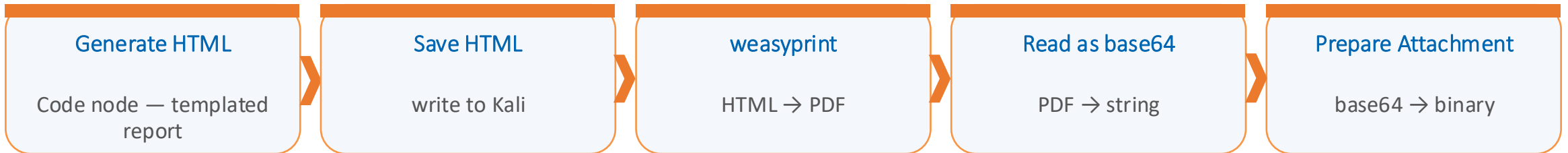
Claude reads the same finding but reasons about access-control intent — and may file it elsewhere.

```
permissions-policy missing → A01 Broken Access Control
```

Neither is 'wrong'. Keep both views — the human adjudicates.

HTML → PDF → inbox

STAGE 5 • REPORTING



```
Prepare Attachment — Code node

const binaryData = Buffer.from($json.stdout, 'base64'); // decode PDF
return [{
  json: { filename: `vulnerability_report_${scanId}.pdf` },
  binary: { data: await this.helpers.prepareBinaryData(
    binaryData, filename, 'application/pdf') }
}];
```

Then, in parallel

 **Gmail** - PDF to the requester

 **Telegram** - instant summary

A Merge re-joins both before the final payload.

The heredoc that silently ate the report

WAR STORY • WHAT BROKE

before — SSH heredoc

```
# Save HTML Report (the naive version)
cat << 'HTMLEOF' > {{ $out }}/{{ $scan_id }}/report.html
{{ $json.html_report }}      # ← large HTML gets TRUNCATED here
HTMLEOF
```

after — binary upload

```
// Prepare HTML Binary → SSH File Upload (the fix)
return [{ json: { scan_id },
  binary: { data: await this.helpers.prepareBinaryData(
    Buffer.from(html, 'utf-8'), 'report.html', 'text/html') } }];
// → upload as a FILE, not a shell command
```

The lesson

Piping a big string through an SSH command hits a command-length limit and truncates — silently. The report was cut off right before the Low-severity section, with no error anywhere.

Fix: stop treating a file as a command. Buffer the HTML into binary and use the SSH node's File-Upload resource. Large payloads belong in files, not argv.

Build Response Payload - one JSON to rule them

STAGE 6 · THE CONTRACT + TELEMETRY

```
Build Response Payload - Code node

return [{ json: {
  scan_id, phase: 'complete', risk_level,           // CRITICAL / HIGH / MEDIUM / LOW
  total_findings, critical_count, high_count, medium_count, low_count,
  sql_injection_detected,                          // drives the dashboard banner
  services,                                         // parsed from nmap with a regex
  findings,                                         // flattened + tagged by tool
  raw_tool_output: { nmap, nuclei, sqlmap, ... },   // all 11 tools, verbatim
  ai_report_markdown                               // the Claude analysis
}}];
```

The contract

One structured object is what the email, the PDF and the dashboard all consume. Define the shape once, and every consumer stays decoupled from the 11 tools that produced it.

Telemetry that never blocks

7 "Report:" HTTP nodes POST each phase to the dashboard — every one set **Retry-On-Fail** + **On-Error-Continue**, so a flaky dashboard can never stall a scan.

The live dashboard

STAGE 6 • VISUALIZATION



A tiny stack - Cloudflare Pages (static HTML/CSS/JS) + one Function + KV. No framework, no build step.

Live terminal log

phase-by-phase as the scan runs

Risk hero + severity

Critical/High/Medium/Low at a glance

Findings + raw output

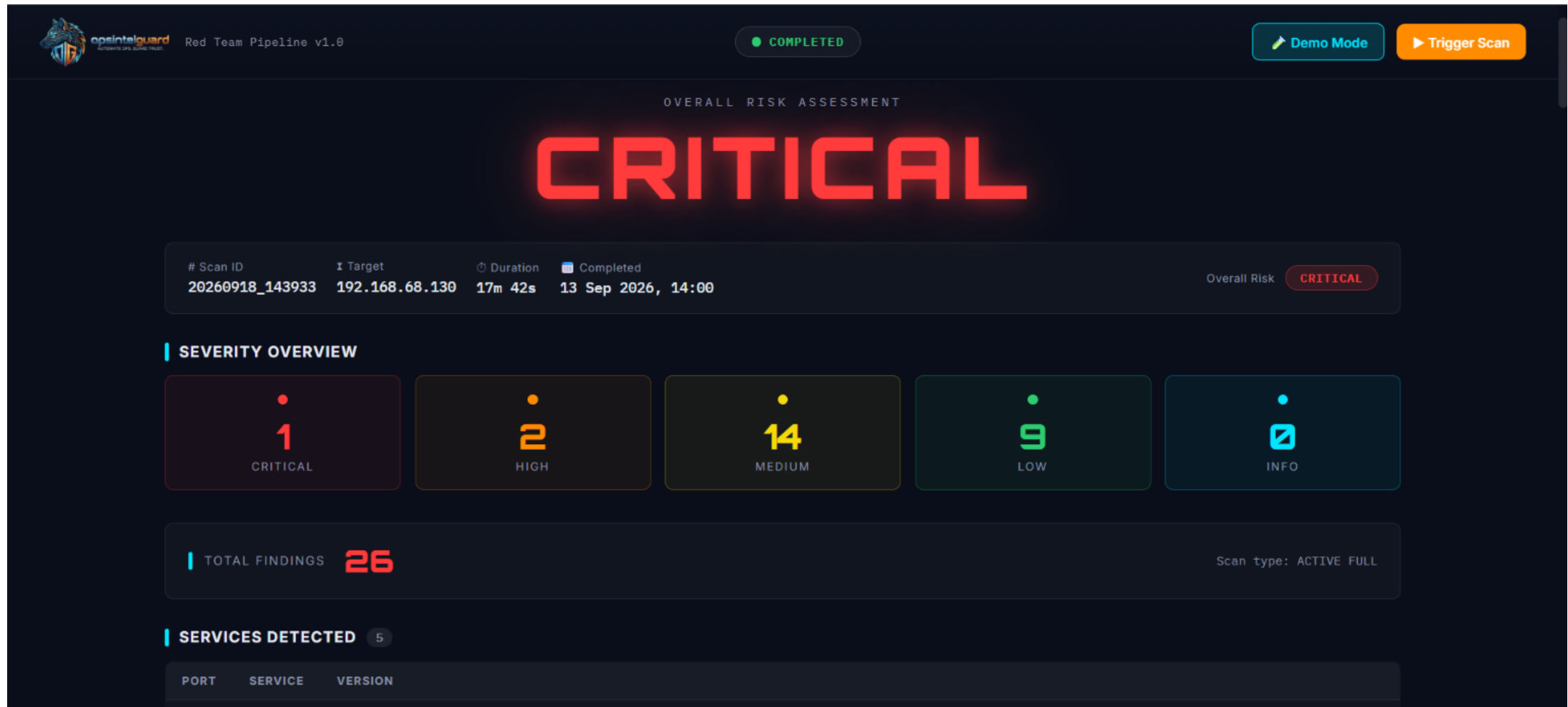
every tool, expandable

AI report, rendered

the Claude markdown, inline

LIVE DEMO

DVWA Red Team Pipeline — trigger → live log → results → AI report



Automated Scanning · AI Analysis · Live Dashboard

Lessons from building it for real

REFLECTION

1

Fail-open beats fail-perfect

safeGet + continue-on-error means a degraded report, never a dead run.

3

Keyword triage is a first pass

Cheap and instant — but CVE data and an AI layer are what make it trustworthy.

5

Determinism wins on stage

A scripted demo that always works beats a live scan that might not.

2

The edge will time out on you

Fast-ack decoupling is mandatory once a job outlives an HTTP connection.

4

Low-code is not no-code

The interesting parts are all JavaScript: aggregation, classification, enrichment.

6

Instrument from day one

Phase telemetry turned an opaque 20-min job into something you can watch.

What I'd change next

WHAT'S NEXT

Finish the report chain

wire the binary-upload fix end-to-end, kill the heredoc for good

Parse, don't keyword-match

a structured parser per tool (or tool JSON output) instead of regex triage

Queue long scans

async job + status polling so full 1–65535 port ranges aren't a stage-time problem

Harden the webhook

auth + rate-limits; right now anyone with the URL can trigger a scan

Containerise the toolbox

pin tool versions in a container instead of a hand-maintained Kali VM

THANK YOU!

Questions?

Happy to go deeper on any node, the AI prompt, the failures — or the demo.

Veronika Stoyanova

Co-founder · Cyber Security Automation

<https://www.linkedin.com/in/veronika-stoyanova/>

Built with

n8n · Kali · Nmap · Nuclei · Nikto ·
SQLmap · Gobuster · ZAP · testssl.sh ·
wafw00f · NVD · Claude · Cloudflare